

EXPERIMENT 1. INTRODUCTION TO DISTRIBUTED SYSTEMS

Topic: Basic Introduction to Distributed Systems

Objective: To provide students with a fundamental understanding of distributed systems, their characteristics, communication mechanisms, and challenges through a simple client-server application.

Activity 1: Basic Client-Server Communication

Create a client-server communication model using **Python** and **Sockets**.

Code:

Server.py (Python):

```
import socket
# Server setup
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.bind(('localhost', 12345))
server_socket.listen(1)
print("Server is waiting for client connection...")
client_socket, addr = server_socket.accept()
print(f'Connection established with {addr}')
# Sending data to client
client_socket.send(b'Hello from server')
# Closing connection
client_socket.close()
server_socket.close()
```

Client.py (Python):

```
import socket
# Client setup
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client_socket.connect(('localhost', 12345))
# Receiving data from server
data = client_socket.recv(1024)
print(f'Received message: {data.decode()}')
# Closing connection
client_socket.close()
```

Task:

1. Run the server code in one terminal.
2. Run the client code in another terminal.
3. Observe how the server sends a message and the client receives it.
4. Record the output displayed on both terminals.

Expected Outcome

1. The server accepts a client connection.
2. The client receives the message "Hello from server".

Activity 2: Modify the Message

Learning Outcome

- Understand how data is transmitted between distributed components.

Task

1. Change the message sent by the server.
2. Run the applications again.
3. Verify that the client receives the updated message.

Activity 3: Two-Way Communication

Learning Outcome

- Understand request-response communication.

Task

Modify the programs so that:

- The client sends a message to the server.
- The server receives and displays the message.
- The server sends a reply back to the client.

Activity 4: Multiple Client Connections

Learning Outcome

- Observe how distributed systems handle multiple users.

Task

1. Modify the server to accept multiple client connections.
2. Open several client terminals and connect simultaneously.

Activity 5: Network Failure Simulation

Learning Outcome

- Understand fault tolerance challenges in distributed systems.

Task

1. Run the client without starting the server.
2. Observe the error generated.
3. Document the exception message.

Activity 6: Communication Delay Experiment

Learning Outcome

- Understand network latency and waiting time in distributed systems.

Task

Add a delay before the server sends a message:

```
import time
time.sleep(5)
```

Run the application and observe the behaviour.

Observation Table

Experiment	Observation	Result
Basic Communication		
Message Modification		
Two-Way Communication		
Multiple Clients		
Server Unavailable		
Communication Delay		
Basic Communication		

Questions

1. What roles do the client and server play in this experiment?
2. What communication protocol is used in this program?
3. Why are sockets important in distributed systems?
4. What happens if the server is not running when the client attempts to connect?
5. How does the server know that a client has connected?
6. What challenges might arise when many clients connect simultaneously?